

Meeting Minutes of B06

Date and Time: 2026/04/30 11:20

Location: T29-101

Present:

- t330016056 Bohan Yang
- t330026083 Xinze Li
- t330025052 Chenxu Liu
- t330026245 Ye Zuo
- t330026244 Kaiwen Zuo
- t330026219 Wenyi Zhang

Next Meeting: 2026/05/14 11:20

Agenda:

1. Review the Lab 9 briefing on mapping class diagrams to source code.
2. Study the unit testing tutorial requirements and decide the programming language/framework direction.
3. Choose one suitable class from the system for unit testing.
4. Plan the required Lab 9 submission package, including source file, test case file, and Word report.
5. Assign responsibilities and confirm the submission deadline.

Announcements:

The teacher explained the document Mapping Diagram to Code, which contains general rules for mapping class diagrams into source code. The group noted that all classes in the restructured class diagram should appear in the code. In addition to participating classes, the implementation may also include control classes and boundary classes.

Participating classes in the class diagram should not directly access control classes or boundary classes, following the object-oriented analysis guidance discussed in Lecture 7. The teacher introduced unit testing and test-driven development (TDD). TDD means writing test cases before implementing code, then running tests during implementation to check whether the code behaves correctly with respect to the test cases. Unit testing concepts are covered in SE Lecture 14 and in the tutorial materials on iSpace.

The Lab 9 tutorial materials include Java/JUnit and IntelliJ IDEA testing resources, Python/Flask unit testing materials, Python class materials, W3Schools Python references, Flask framework demo videos, Flask unit test videos, and an older Django quick start document for students who prefer Django. The teacher suggested that teams use Python/Flask by default unless they have a good reason to choose another framework. Java/Spring Boot, Django, and AI tools such as LLMs or coding agents were also mentioned as possible options, but AI tools were described as high-reward and high-risk because the team would need to learn them independently.

For Lab 9, each team must choose one class from its software system and implement unit testing for that class. The chosen class should have enough meaningful methods other than getters and setters. The programming language used for unit testing should be the same as the language used for coding. Each team should submit a zip file containing:

- One source file class from the system for testing.
- One source file test case class containing the test case functions and all source files required to run the test case.
- One Word file containing the class under testing, a snapshot of the testing results, and the testing methods used, such as equivalence class testing or boundary testing.

Submission requirement discussed in class: each team should submit the required files to both iSpace and GitLab before 23:59, Saturday 2 May 2026. Contribution levels on the cover should be marked as Full, Fair, Little, or None next to each member's name.

The teacher also announced that there will be no formal lab meeting on 7 May 2026. TAs will hold online tutorials to answer questions about tutorials, programming languages, frameworks, coding, and testing. The meeting ID will be announced in the WeCom group. Students should prepare questions in advance.

Discussion:

1. The group first reviewed the mapping-from-design-to-code guidance and agreed that the implementation should be traceable to the restructured class diagram. Members agreed to avoid creating source code that conflicts with the design documents unless the documents are updated consistently.
2. The team discussed the distinction between participating classes, control classes, and boundary classes. Members agreed that participating/domain classes should focus on system data and behavior, while UI and control logic should be separated to keep the code structure consistent with the object-oriented design.
3. For language and framework selection, the team agreed to follow the teacher's default recommendation and use Python/Flask unless further technical investigation shows a stronger reason to choose another option. The group considered this choice safer because the team has already seen Flask in the Week 1 GitLab demo and Flask is simpler to learn than Django.
4. The group agreed not to make AI/coding agents the main implementation path for the whole team at this stage because of the schedule risk. Members noted that one or two people may still explore AI tools for support, but the main coding and testing work should continue with a stable framework choice.
5. For the unit testing target, the team agreed to choose one class with meaningful validation or business logic instead of a class containing only getters and setters. The selected class should allow the team to demonstrate clear testing methods such as equivalence class testing and boundary testing.
6. The members agreed that two or three people should focus on the unit testing deliverable for this week, while the rest of the team starts or continues coding other

system components. This division allows the team to satisfy the Lab 9 submission requirement while also preparing for upcoming coding and testing work.

7. Responsibility allocation for this week:

- Yang Bohan: write Meeting Minutes 8, integrate the Lab 9 submission package, and check iSpace/GitLab upload readiness.
- Li Xinze: prepare the Word report structure, including class-under-test description, testing methods, and test result snapshot section.
- Liu Chenxu: select and implement the source class for unit testing, ensuring it contains meaningful non-getter/setter methods.
- Zuo Ye: coordinate progress, confirm framework choice, and run the final checklist before submission.
- Zuo Kaiwen: write or refine the test case class and verify that all required source files can run together.
- Zhang Wenyi: document the testing methods used and support result screenshot collection and formatting.

Any Other Business (AOB):

The group agreed to complete the unit testing deliverable before the 2 May 2026 deadline and submit early enough to avoid upload problems. The team will prepare questions for the TA online tutorial if issues arise when using Flask, writing test cases, or mapping the design classes into source code. Members also noted that the following weeks will focus more heavily on coding and testing, so implementation decisions made this week should remain simple, consistent, and easy to maintain.